

Memory-Augmented SmolVLA: Brief Report

memvla_libero, condensed

tarcode2004 / aleksantari

May 2026

1. Headline result

Configuration	spatial	object	goal	libero_10	Overall
V2 baseline (no memory, 100K)	84.0	99.0	96.0	72.0	87.75
memvla v1, (num_groups=4, group_size=8) @ 100K	74.0	96.0	79.0	44.0	73.25
memvla v2, (num_groups=12, group_size=4) @ 65K (<i>in progress</i>)	84.0	97.0	94.0	52.0	81.75

+8.5pp overall by changing only (num_groups, group_size); same model, same mem_length=4, **zero ToMe consolidations during training**. v2 matches baseline exactly on libero_spatial (84.0); within 2pp on object and goal; gate active at $\alpha \approx 0.504$. **libero_10 is the residual: 52 vs baseline 72 (−20pp)**, but v2 is still climbing at 65K of a planned 100K.

Weak claim (“memory can be added without breaking the policy”) is **supported**. **Strong claim** (“memory improves long-horizon performance”) is **still open** pending 100K and downstream experiments.

2. Why this problem

VLAs like SmolVLA are Markovian: each frame is processed independently with no episodic context. This is fine for pick-and-place but breaks on long-horizon tasks (*open drawer, place item, close drawer*) where the right action depends on what already happened.

LIBERO’s libero_10 is the long-horizon benchmark, and it is the suite where the V2 baseline has the most headroom (28pp below the paper-implied ceiling of 100). Memory-augmented policies *should* help here most.

MemoryVLA ([arXiv:2508.19236](https://arxiv.org/abs/2508.19236)) adds a CLS-token memory bank between VLM and action head with ToMe consolidation and reports +26pp on long-horizon real-world tasks vs CogACT. Our project ports that idea to SmolVLA’s 450M architecture.

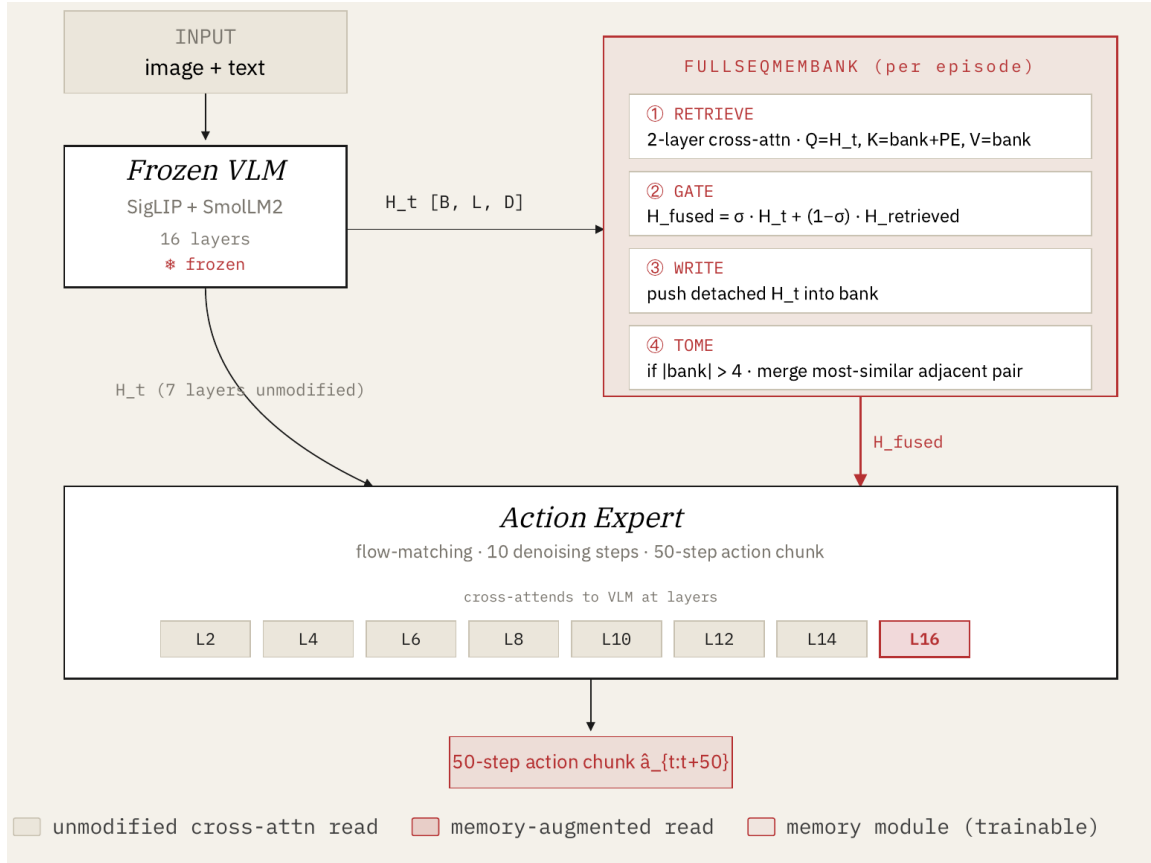


Figure 1: FullSeqMemBank pipeline. Memory module sits between the frozen VLM and the action expert: retrieve via 2-layer cross-attention with sinusoidal timestep PE on bank keys, gate-fuse current and retrieved features with a per-token sigmoid, write detached H_t to a per-episode bank, ToMe-merge most-similar adjacent pair when bank > 4 entries. Memory is read by the action expert only at L16; the other 7 cross-attention reads see unmodified VLM features.

3. Architecture

SmolVLA backbone (frozen). SigLIP vision encoder + first 16 layers of SmolVLM2’s text model (32 native, truncated via `text_model.layers[:16]`). The action expert is paired 1:1 with the VLM and reads VLM K,V at every even operative layer (L2, L4, ..., L16); eight cross-attention handoffs, no single post-VLM seam.

Memory module (trainable, 23M params).

- **Bank.** Per-episode dictionary; each entry stores the full prefix [$L \approx 170$, $D=576$]. Capacity `mem_length=4`. ToMe consolidation merges the most-similar adjacent pair on overflow.
- **Retrieval.** Two stacked CrossTransformerBlocks; sinusoidal TimestepEmbedder added to bank keys only.
- **Gate.** Per-token sigmoid on [`current`; `retrieved`]; `std=1e-3` init so $\alpha \approx 0.5$ at step 0. Fusion: $\alpha \cdot \text{current} + (1 - \alpha) \cdot \text{retrieved}$.

Injection point: L16 with `inject_before=True`; modifies the residual stream just before the last operative VLM layer, so both L16’s VLM self-attention and the L16 expert cross-attention (the final handoff) see memory-fused features. The other seven cross-attention reads (L2–L14) see vanilla VLM features. We accept this 1/8 signal dilution for a single-point, minimally invasive modification.

Action expert (trainable, 98M params). Reinitialized from scratch when loading `lerobot/smolvla_base` (only VLM + SigLIP weights kept), matching V2’s from-scratch action-expert protocol. Flow-matching with 10 denoising steps over 50-step action chunks.

Trainable / total: 121M / 473M (~26%).

4. Procedure

Training data. HuggingFaceVLA/libero (all 4 suites combined). `GroupedEpisodeLoader` yields `num_groups × group_size contiguous` frames per batch with per-frame `episode_ids` and `timesteps`. Memory training requires contiguous windows because the bank must be built in temporal order.

Two configurations. v1 used (`num_groups=4`, `group_size=8`) = batch 32; 4 distinct episodes per gradient step. v2 used (`num_groups=12`, `group_size=4`) = batch 48; 12 distinct episodes per gradient step.

Hyperparameters (mirror V2 baseline): AdamW, peak LR $1e-4 \rightarrow 2.5e-6$ cosine, 1K warmup, `weight_decay=1e-10`, `max_grad_norm=10`, AMP bfloat16, 100K target steps, image augmentations on. `n_action_steps=10` at inference.

Eval. LiberoEnv per-episode instantiation, suite-specific `_max_episode_steps` (280 / 280 / 300 / 520 for `libero_10`), 10 episodes/task $\times$ 10 tasks $\times$ 4 suites = **400 rollouts per checkpoint**. Bypass ablation forces $\alpha = 1$ on the same checkpoint to isolate “what the trained expert would do without memory at inference.”

5. Results

v1 was net-harmful vs both baseline (−14.5pp overall) and its own bypass ablation (−2.75pp). The bypass run on the v1 checkpoint scored 76.00 / 73.25; memory *hurt* on top of an already-underperforming expert.

v2 fixes most of the gap with a single config knob. Same architecture, same `mem_length=4`, but 12 distinct episodes per gradient step instead of 4. At 65K of a planned 100K:

Suite	Δ vs v1	Δ vs baseline
spatial	+10.0	0.0 (match)
object	+1.0	−2.0
goal	+15.0	−2.0
libero_10	+8.0	− 20.0

Suite	Δ vs v1	Δ vs baseline
Overall	+8.5	−6.0

Three observations. (1) Gate stayed active throughout v2 ($\alpha \approx 0.504$ per-suite: 0.505 / 0.504 / 0.504 / 0.502); memory is being used. (2) v2 ran with `group_size = mem_length = 4`, so the bank fills exactly at the end of every group and **ToMe never fires during training**; the gain cannot come from improved bank-depth alignment. (3) `libero_10` is still trailing; but the training trajectory is still rising at 65K, and a 100K result is queued.

6. Takeaway: gradient diversity was the bottleneck

SmolVLA, in our reproduction runs, fails to converge at batch ≤ 8 , only starts converging at 16, and reproduces paper-grade numbers at 32. The original paper trained at 64. Larger batches converge better on this architecture.

Memory training compounds the demand: each `group_size`-frame group consumes slots on one episode, so the relevant metric is **distinct episodes per gradient step**, not raw batch size.

The episode-diversity ladder.

Run	Distinct episodes per gradient step	Ratio vs baseline
V2 baseline (random sampling, batch 32)	32	1.0 $\times$ (reference)
memvla v1 (4 groups $\times$ 8)	4	0.125 $\times$ (8 $\times$ less)
memvla v2 (12 groups $\times$ 4)	12	0.375 $\times$ ($\sim 2.7\times$ less)

v1’s 4 episodes/step was equivalent in diversity to a batch=4 baseline run; well below SmolVLA’s empirical convergence floor. The memory architecture was being trained at a regime where the no-memory baseline itself would not have converged stably. v2’s 12 episodes/step brings diversity into a viable regime; still $\sim 2.7\times$ short of baseline, but enough to close most of the gap.

Reinterpreting v1’s “memory hurts” reading. v1’s bypass ablation looked like memory was actively breaking the policy. In light of v2, the bypass result was **downstream of under-training due to insufficient gradient diversity, not a property of the memory pathway**. Memory itself is not broken; v1 simply could not learn a useful retrieval policy from 4 episodes per gradient step.

Caveat (comparison axis). v2 ran at batch 48 vs the V2 baseline’s batch 32. At 65K steps, v2 has seen $65K \times 48 = 3.12M$ frame-gradients; the V2 baseline at 65K would have seen 2.08M. Two valid framings: (a) same-steps comparison (v2 has $1.5\times$ the data per step; the natural partner is baseline @ 60K, queued for evaluation), or (b) same-data-volume comparison (v2 @ 65K $\approx$ baseline @ 100K at $\sim 3.2M$ frames; that’s the 81.75 vs 87.75 comparison above). The most conservative reading uses (b).

7. Next experiments

In priority order:

1. **Finish v2 to 100K and re-eval.** Read the slope on `libero_10` between 65K and 100K. Continuing climb $\rightarrow$ under-training was the residual; plateau $\rightarrow$ architectural fixes needed.
2. **Bypass eval on the v2 checkpoint.** v1’s bypass showed memory was net-harmful; v2’s is not yet measured. If (`mem-on` — `bypass`) is positive, retrieval + gating are doing useful work given enough diversity. Cleanest evidence of “memory contributes” we can produce.
3. **Push diversity higher.** (`num_groups=16`, `group_size=4`) at batch 64 closes more of the 12 $\rightarrow$ 32 gap. (`num_groups=32`, `group_size=1`, `mem_length=1`) at batch 32 is a max-diversity-zero-memory floor for sanity-checking the diversity axis.

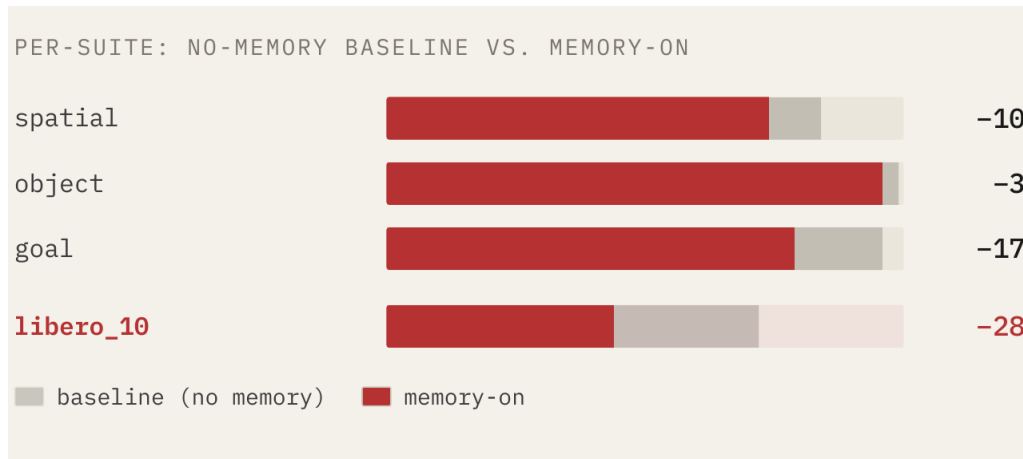


Figure 2: Per-suite success rates: V2 baseline (grey) vs memvla v1 memory-on (red). v1 underperformed every suite; biggest regression was on libero_10 (−28pp), the suite memory was supposed to help most. v2 (not pictured) brings spatial/object/goal back to within 2pp of baseline; libero_10 is still −20pp.

4. **MemoryVLA-style sparse sampling.** Their GroupRLDSDataset (verified from `vla/datasets/rlds/dataset.py:7`) randomly subsamples `group_size` frame indices across the trajectory and sorts by timestep; non-contiguous, full-episode-span training banks. The contiguous-window asymmetry we currently have may be the source of the residual libero_10 gap. One-loader change, no model changes.

8. References

Paper (year)	arXiv	Relevance
SmolVLA (2025)	2506.01844	Base model. Architecture, LIBERO Table 13.
MemoryVLA (2025)	2508.19236	Source design we port. CLS-token cognitive bank, ToMe, sigmoid gate, sinusoidal timestep PE.
ContextVLA (2025)	2510.04246	Alternative single-token-per-frame design; candidate baseline.
LeRobot SmolVLA (2025)	lerobot	<code>embed_prefix</code> , <code>SmolVLMWithExpertModel.forward</code> .

For full procedure, drawback analysis, bypass-ablation framing, and V1-iteration history, see `report.md` (24 pages).